



LeetCode 2352: Equal Row and Column Pairs

1. Problem Title & Link

Title: LeetCode 2352: Equal Row and Column Pairs

Link: <https://leetcode.com/problems/equal-row-and-column-pairs/>

2. Problem Statement (Short Summary)

You are given an:

$n \times n$ matrix

Count the number of pairs:

(row, column)

such that:

row values == column values

in exactly the same order.

Return the total count.

3. Examples (Input → Output)

Example 1

Input:

```
grid = [  
  [3,2,1],  
  [1,7,6],  
  [2,7,7]  
]
```

Output:

1

Explanation:

Row:

[2,7,7]

matches Column:

[2,7,7]

Example 2

Input:

```
grid = [  
  [3,1,2,2],  
  [1,4,4,5],  
  [2,4,2,2],  
]
```



```
[2,4,2,2]  
]
```

Output:

3

Example 3

Input:

```
grid = [  
  [1,1],  
  [1,1]  
]
```

Output:

4

Explanation:

Every row matches every column.

4. Constraints

- $1 \leq n \leq 200$
- $1 \leq \text{grid}[i][j] \leq 10^5$

Need better than:

$O(n^3)$

if possible.

5. Core Concept (Pattern / Topic)

HashMap / Counting

Related Topics:

- Matrix Traversal
- Tuple Hashing
- Frequency Counting

6. Thought Process (Step-by-Step Explanation)

Brute Force

Compare:

Every row

with

Every column

For each comparison:



Check n elements

Complexity:

$O(n^3)$

Because:

n rows

× n columns

× n comparisons

Optimized Thinking

Store all rows inside a HashMap.

Example:

$(3,2,1) \rightarrow 1$

$(1,7,6) \rightarrow 1$

$(2,7,7) \rightarrow 1$

Now build each column.

If column exists in HashMap:

answer += frequency

This avoids repeatedly comparing arrays.

Why It Works

Rows and columns are sequences.

Sequences can be stored as:

tuple

which becomes hashable.

Hash lookup:

$O(1)$

average time.

7. Visual / Intuition Diagram (ASCII Diagram)

Matrix:

3 2 1

1 7 6

2 7 7

Rows:

$(3,2,1)$

$(1,7,6)$

$(2,7,7)$



```
Columns:
(3,1,2)
(2,7,7) ← Match
(1,6,7)
Answer:
1
```

8. Pseudocode (Language Independent)

```
Create hashmap

For each row:
    store row frequency

answer = 0

For each column:
    build column tuple
    answer += hashmap[column]

Return answer
```

9. Code Implementation

 **Python**

```
from collections import Counter

class Solution:
    def equalPairs(self, grid):
        rowMap = Counter(tuple(row) for row in grid)

        n = len(grid)

        answer = 0

        for col in range(n):
            column = tuple(
                grid[row][col]
                for row in range(n)
            )

            answer += rowMap[column]
```



```
return answer
```

✓ Java

```
import java.util.*;

class Solution {

    public int equalPairs(int[][] grid) {

        int n = grid.length;

        Map<String, Integer> map =
            new HashMap<>();

        for (int[] row : grid) {

            String key =
                Arrays.toString(row);

            map.put(
                key,
                map.getOrDefault(key, 0) + 1
            );
        }

        int answer = 0;

        for (int col = 0; col < n; col++) {

            int[] column =
                new int[n];

            for (int row = 0; row < n; row++) {

                column[row] =
                    grid[row][col];
            }

            String key =
                Arrays.toString(column);

            answer +=
                map.getOrDefault(key, 0);
        }

        return answer;
    }
}
```



10. Time & Space Complexity

Time Complexity

Building row map:

$O(n^2)$

Building columns:

$O(n^2)$

Overall:

$O(n^2)$

Space Complexity

$O(n^2)$

HashMap stores rows.

11. Common Mistakes / Edge Cases

Mistake 1

Using lists as HashMap keys in Python.

Wrong:

```
dict[[1,2,3]]
```

Lists are mutable.

Use:

```
tuple(row)
```

Mistake 2

Comparing sets instead of sequences.

Order matters.

Mistake 3

Not handling duplicate rows.

Need frequency counting.

12. Detailed Dry Run (Step-by-Step Table)

Input:

```
grid = [  
    [3,2,1],  
    [1,7,6],  
    [2,7,7]
```



]

Row Map:

Row	Frequency
(3,2,1)	1
(1,7,6)	1
(2,7,7)	1

Columns:

Column	Exists?
(3,1,2)	No
(2,7,7)	Yes
(1,6,7)	No

Answer:

1

13. Common Use Cases (Real-Life / Interview)

Data Validation

Check if records match across dimensions.

Spreadsheet Analytics

Compare rows and columns for consistency.

Matrix Pattern Detection

Identify repeated structures.

Database Matching

Cross-check row and column signatures.

14. Common Traps (Important!)

- Forgetting tuple conversion
- Ignoring duplicates
- Comparing unordered collections
- Wrong column construction

15. Builds To (Related LeetCode Problems)

1. LC 49 — Group Anagrams



2. LC 242 — Valid Anagram
3. LC 560 — Subarray Sum Equals K
4. LC 1207 — Unique Number of Occurrences
5. LC 2352 — Equal Row and Column Pairs

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^3)$	$O(1)$	Slow
Trie	$O(n^2)$	$O(n^2)$	Complex
HashMap + Tuple	$O(n^2)$	$O(n^2)$	Optimal

17. Why This Solution Works (Short Intuition)

Each row is stored as a hashable sequence. Every column is converted into the same representation and checked in constant time against the stored rows. Frequency counting automatically handles duplicate matches.

18. Variations / Follow-Up Questions

Follow-Up 1

What if matrix is:

$m \times n$

instead of square?

Still possible using the same HashMap approach.

Follow-Up 2

What if rows change frequently?

Use rolling hashes for efficient updates.

Follow-Up 3

What if we need matching diagonals too?

Store diagonals using the same tuple-hashing technique.